Command Design Pattern

The Command Design Pattern is a behavioural design pattern that encapsulates a request as an object, thereby decoupling the sender of the request from the receiver and allowing flexible execution of operations.

- Turns a request into an object so it can be passed, stored, queued, or undone easily.
- Encapsulates the request as an object and decouples the invoker (sender) from the receiver (handler).

Example: *A Remote Controller where a user sends commands like turning on lights, TV, speakers, or AC. The remote does not directly control the devices; instead, it passes command objects to them. Each device then executes the command independently, keeping the sender and receiver loosely coupled.*

Real Life Software Applications

Some real-life software examples where the Command Pattern is commonly used are:

- **GUI Buttons and Menu:** Each button or menu item (Copy, Paste, Save, Undo) is a command object. The framework doesn't know the exact action — it just executes the command.
- **Job Queues:** AWS SQS, RabbitMQ, Each job/task is wrapped in a command object. Queues hold commands and execute them later or distribute them across systems.

Components

The Command Design Pattern consists of the following key components that work together to decouple the sender and receiver of a request

- **Command Interface:** Declares the common method (e.g., execute()) for all commands.
- **Concrete Commands:** Implement the interface and encapsulate specific actions (e.g., turn on TV).
- **Invoker:** Initiates the command execution without knowing the details (e.g., remote control).
- **Receiver:** Performs the actual operation defined by the command (e.g., TV, stereo).

Working

Using Remote Control Example

- The remote control acts as the Invoker
- Each button is a Command object
- The device (TV, AC, Light) is the Receiver
- When a button is pressed, the command's execute() method is called, which triggers the action on the receiver

Uses

This pattern is used when a request needs to be encapsulated as an object for flexible execution.

- Implementing remote controls and button-based actions
- Supporting undo and redo operations (editors, drawing tools)
- Logging and storing command history.

The Command Design Pattern

The **Command Design Pattern** turns a request into a standalone object. This separate (decouples) the object that *sends* the request from the object that *handles* it, allowing you to easily pass, queue, log, or undo actions.

💡 The Big Picture: A TV Remote Control

Think of a **Universal Remote Control**:

- The remote doesn't know *how* a TV turns on or *how* an AC changes temperature.
- It just knows that when you press a button, it triggers a **Command**.
- The device (**Receiver**) receives the command and does the actual work.

🧩 The 4 Core Components

The pattern works by splitting responsibilities into four parts:

Component	What it does	Remote Control Example
1. Command Interface	An interface defining a basic method like execute().	The concept of "pressing a button".
2. Concrete Command	The specific action being requested.	The "Turn on TV" command.
3. Invoker	The trigger that starts the action.	The **Remote Control** itself.
4. Receiver	The thing that actually performs the action.	The **TV** or **AC**.

💻 Real-World Software Uses

- **GUI Buttons & Menus:** In apps, buttons like "Copy", "Paste", or "Save" are individual command objects. The app UI doesn't need to know how "Paste" works; it just triggers the command.
- **Job Queues:** Systems like AWS SQS or RabbitMQ wrap tasks into command objects so they can be queued up, delayed, or spread across different servers.
- **Undo/Redo Actions:** Because commands are saved as objects, a system can keep a history of them to easily reverse or re-run operations (like in text or drawing editors).